

Chapitre 5 – Procédures stockées

I.	Objectifs	1
II.	Intérêts.....	1
III.	Utilisation avec SQL Server.....	2
1.	Création de la base Comptoir	2
2.	Détachement de la base Comptoir	2
3.	Attacher la base de données	2
4.	Créer une procédure stockée sans paramètre	3
5.	Exécuter une procédure stockée sans paramètre en C#.....	3
6.	Créer une procédure stockée avec paramètre.....	4
7.	Exécuter une procédure stockée avec paramètre en C#	5
IV.	Application à EpokaAbsences.....	5
1.	Classe SalarieDAO	5
2.	Classe AbsenceDAO	7
3.	Classe MotifDAO.....	7
V.	Conclusion	8
VI.	PPE	8

I. Objectifs

Les procédures stockées répondent à deux objectifs :

- Elles permettent d'élargir les fonctionnalités d'une base de données, en ayant des éléments de programmation au sein même de la base de données, accessibles par toute application (des droits existent) ;
- Elles sont un élément de sécurité puisque on peut obliger à les utiliser plutôt que de donner accès aux tables et requêtes, ce qui **centralise** la façon d'accéder aux données ;

Tous les SGBD ne possèdent pas de procédures stockées, mais les poids lourds sont là :

- MySQL (depuis la version 5) ;
- SQL Server ;
- Oracle ;
- DB2 ;
- PostgreSQL ;
- ...

II. Intérêts

- Simplification : Le code est **plus simple à comprendre** ;
- Rapidité / Performance : La procédure stockée est **compilée** ;
- Sécurité : Les applications et les utilisateurs n'ont **aucun accès direct aux tables**, mais passent par des procédures stockées prédéfinies. **Injection SQL impossible** ;

Le code est constitué de SQL, enrichi avec un langage de programmation procédural simple, qui diffère très peu selon les SGBD :

- SQL Server utilise le *Transac SQL* ;
- MySQL utilise ... *MySQL* ;

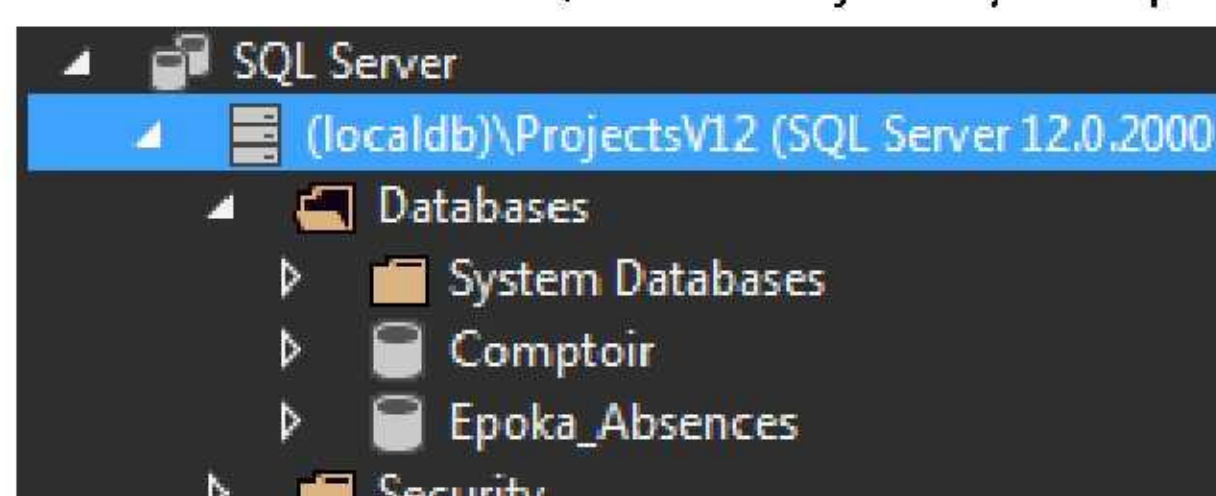
III. Utilisation avec SQL Server

Des procédures stockées existent déjà. Parmi elles, le détachement et l'attachement d'une base.

1. Création de la base Comptoir

Dans Visual Studio, à partir de la fenêtre *SQL Server Object Explorer* :

- Créer une nouvelle requête (*New Query*) sur la bonne instance de *SQL Server*, et exécuter le script de création de la base *Comptoir*, **en adaptant la localisation des fichiers** ;
- Utiliser le bouton  sur le dessus de la fenêtre pour l'exécuter ;
- Actualiser la fenêtre *SQL Server Object Explorer* pour voir la base créée :



2. Détachement de la base Comptoir

Pour détacher une base, et avoir accès aux fichiers (déplacement, ...), on utilise la procédure système *sp_detach_db*.

La documentation de *SQL Server* indique :

```
sp_detach_db [ @dbname= ] 'database_name'  
    [ , [ @skipchecks= ] 'skipchecks' ]  
    [ , [ @keepfulltextindexfile = ] 'KeepFulltextIndexFile' ]
```

Les noms de paramètres sont précédés de @, leur nom n'est pas obligatoire, le nom de la base est indispensable.

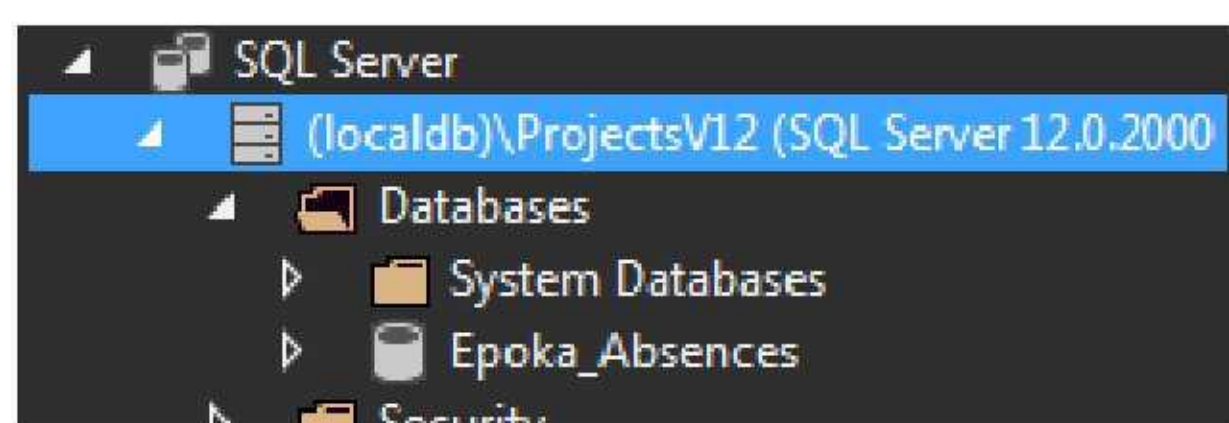
Pour détacher *Comptoir*, taper dans une fenêtre *Query* :

```
sp_detach_db "Comptoir";
```

L'exécution produit :

```
Command(s) completed successfully.
```

Après actualisation, la fenêtre *SQL Server Object Explorer* indique :



3. Attacher la base de données

Extrait de la documentation de *SQL Server* :


```
sp_attach_db [ @dbname= ] 'dbname'
, [ @filename1= ] 'filename_n' [ ,...16 ]
```

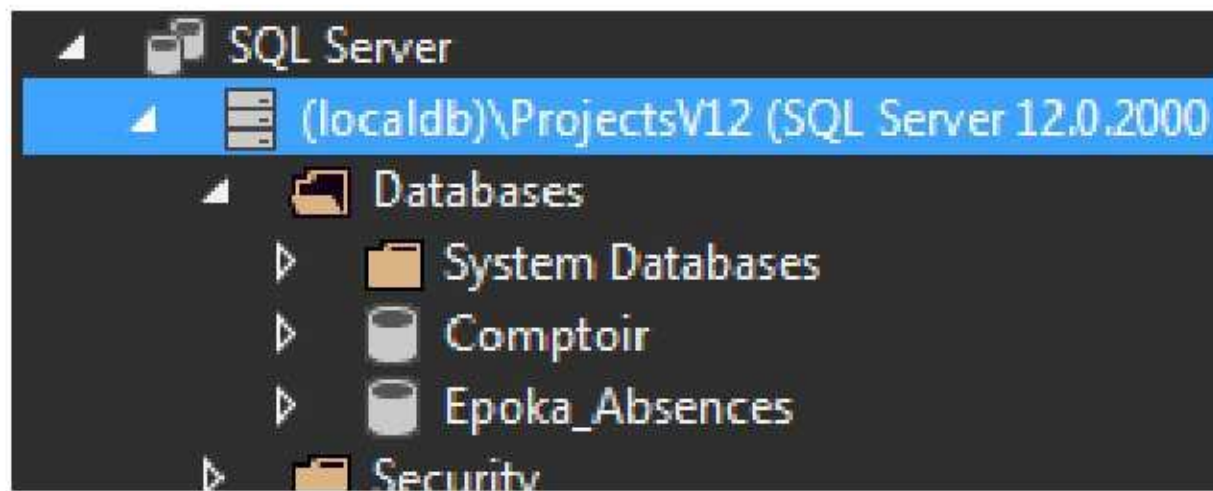
Attacher la base de données *Comptoir* :

```
sp_attach_db "Comptoir",
"D:\Utilisateurs\Francois\Documents\Visual Studio 2013\Projects\BD\comptoir.mdf",
"D:\Utilisateurs\Francois\Documents\Visual Studio 2013\Projects\BD\comptoir.ldf";
```

Ce qui produit :

```
Command(s) completed successfully.
```

Après actualisation, la fenêtre *SQL Server Object Explorer* indique :



4. Créer une procédure stockée sans paramètre

Dans la base *Comptoir*, on va créer une procédure stockée qui affichera tous les produits :

- Créer une procédure stockée (via *Programmability, Stored Procedures*) nommée *listeProduits* ;
- Supprimer les paramètres proposés par défaut ;
- Insérer la requête SQL : `SELECT * FROM Produits` ;
- Définir un code retour (facultatif) indiquant une exécution correcte : 0 ;

```
CREATE PROCEDURE [dbo].[listeProduits]
AS
    SELECT * FROM Produits
RETURN 0
```

- Cliquer sur *Update*, puis *Update Database* pour enregistrer la procédure stockée ;
- Fermer la fenêtre de création de la procédure stockée ;

Pour exécuter la procédure stockée :

- Créer une nouvelle requête ;
- Y taper le nom de la procédure stockée :

```
listeProduits
```

- Exécuter. C'est tout :

	ProRef	ProNom	ProNoFournisseur	ProCodeCategorie	ProQuantiteParU
1	1	Chai	1	1	10 boîtes x 20 sa
2	2	Chang	1	1	24 bouteilles (1 l
3	3	Aniseed Syrup	1	2	12 bouteilles (55
4	4	St. Francis Cider	2	2	48 x 1/2 l

5. Exécuter une procédure stockée sans paramètre en C#

Très peu de différence avec une requête :

- Ouvrir la connexion vers la base ;
- Créer la commande SQL avec la procédure stockée ;
- Facultatif : Indiquer qu'il s'agit d'une procédure stockée ;
- Exploiter le résultat de la requête ;


```

static void Main(string[] args)
{
    SqlConnection connexion = new SqlConnection();
    connexion.ConnectionString = "Data Source=(localdb)\\ProjectsV12; "
        + "Initial Catalog=Comptoir; Integrated Security=SSPI;";
    connexion.Open();
    SqlCommand sqlCmd = new SqlCommand("listeProduits", connexion);
    sqlCmd.CommandType=CommandType.StoredProcedure;

    SqlDataReader sqlDataReader = sqlCmd.ExecuteReader();
    while (sqlDataReader.Read())
    {
        int proRef = (int)sqlDataReader["ProRef"];
        String proNom=(String)sqlDataReader["ProNom"];
        int proUnitesEnStock=(int)sqlDataReader["ProUnitesEnStock"];
        Console.WriteLine (proRef.ToString()+" "+proNom+" "+proUnitesEnStock.ToString());
    }
    sqlDataReader.Close();
    connexion.Close();
    Console.ReadLine();
}

```

Résultat :

```

1 Chai 39
2 Chang 17
3 Aniseed Syrup 13
4 Chef Anton's Cajun Seasoning 53
5 Chef Anton's Gumbo Mix 0
6 Grandma's Boysenberry Spread 120
7 Uncle Bob's Organic Dried Pears 15
8 Northwoods Cranberry Sauce 6

```

6. Créer une procédure stockée avec paramètre

On va créer une procédure stockée qui affichera les produits d'un code catégorie :

- Créer une procédure stockée nommée *listeProduitsDeCategorie* ;
- Garder un paramètre nommé *codeCategorie* de type entier sans valeur par défaut ;
- Insérer la requête SQL : `SELECT * FROM Produits WHERE ProCodeCategorie=@codeCategorie` ;
- Définir un code retour (facultatif) indiquant une exécution correcte : 0 ;

```

CREATE PROCEDURE [dbo].[listeProduitsDeCategorie]
    @codeCategorie int
AS
    SELECT * FROM Produits WHERE ProCodeCategorie=@codeCategorie
RETURN 0

```

- Cliquer sur *Update*, puis *Update Database* pour enregistrer la procédure stockée ;
- Fermer la fenêtre de création de la procédure stockée ;

Pour exécuter la procédure stockée :

- Créer une nouvelle requête ;
- Y taper le nom de la procédure stockée avec son paramètre :

```
listeProduitsDeCategorie @codeCategorie=1
```

- Exécuter :

	ProRef	ProNom	ProNoFournisseur	ProCodeCategorie	ProQuantiteParUn
1	1	Chai	1	1	10 boîtes x 20 sacs
2	2	Chang	1	1	24 bouteilles (1 litre)
3	24	Guaraná Fantástica	10	1	12 canettes (355 ml)
4	34	Sasquatch Ale	16	1	24 bouteilles (70 cl)

7. Exécuter une procédure stockée avec paramètre en C#

Le passage de paramètre demande un objet spécifique. Il ne se fait pas sur la ligne comme avec SSDT (SQL Server Data Tools) :

- Ouvrir la connexion vers la base ;
- Créer la commande SQL avec la procédure stockée ;
- Facultatif : Indiquer qu'il s'agit d'une procédure stockée ;
- Créer autant de paramètres que nécessaire (indique le nom avec le @ et le type) ;
- Facultatif : Indiquer la direction (entrée ou sortie) – Entrée par défaut ;
- Indiquer la valeur (si paramètre en entrée) ;
- Exploiter le résultat de la requête ;

```
static void Main(string[] args)
{
    SqlConnection connexion = new SqlConnection();
    connexion.ConnectionString = "Data Source=(localdb)\\ProjectsV12; "
        + "Initial Catalog=Comptoir; Integrated Security=SSPI;";
    connexion.Open();
    SqlCommand sqlCmd = new SqlCommand("listeProduitsDeCategorie", connexion);
    sqlCmd.CommandType=CommandType.StoredProcedure;
    SqlParameter codeCategorie = sqlCmd.Parameters.Add("@codeCategorie", SqlDbType.Int);
    codeCategorie.Direction = ParameterDirection.Input;
    codeCategorie.Value = 1;

    SqlDataReader sqlDataReader = sqlCmd.ExecuteReader();
    while (sqlDataReader.Read())
    {
        int proRef = (int)sqlDataReader["ProRef"];
        String proNom=(String)sqlDataReader["ProNom"];
        int proUnitesEnStock=(int)sqlDataReader["ProUnitesEnStock"];
        Console.WriteLine (proRef.ToString()+" "+proNom+" "+proUnitesEnStock.ToString());
    }
    sqlDataReader.Close();
    connexion.Close();
    Console.ReadLine();
}
```

Résultat :

```
1 Chai 39
2 Chang 17
24 Guaraná Fantástica 20
34 Sasquatch Ale 111
35 Steeleye Stout 20
38 Côte de Blaye 17
39 Chartreuse verte 69
43 Ipoh Coffee 17
67 Laughing Lumberjack Lager 52
70 Outback Lager 15
75 Rhönbräu Klosterbier 125
76 Lakkaikööri 57
```

IV. Application à EpokaAbsences

L'avantage du Design Pattern DAO, c'est qu'on a uniquement besoin d'intervenir sur les classes DAO.

1. Classe SalarieDAO

La 1^{ère} méthode *lireLesSalaries* contient une requête SQL, à transformer en procédure stockée :

- Créer la procédure stockée *listeSalaries* sans paramètre avec comme SQL : SELECT * FROM Salarie :


```
CREATE PROCEDURE [dbo].[listeSalaries]
AS
    SELECT * FROM Salarie
RETURN 0
```

- L'enregistrer dans la base (*Update, Update Database*) ;
- Utiliser la procédure stockée dans le C# à la place du SQL :

```
try {
    SqlCommand sqlCmd = new SqlCommand("listeSalaries", accesSGBD.connexion);
    sqlCmd.CommandType = CommandType.StoredProcedure;
    SqlDataReader sqlDataReader = sqlCmd.ExecuteReader();
```

- Vérifier le bon fonctionnement

Pour la méthode *ajouterSalarie*, la procédure stockée doit utiliser des paramètres :

- Créer la procédure stockée *ajoutSalarie* ;
- Indiquer 2 paramètres obligatoire typés comme les colonnes de la base ;
- Placer le code SQL : INSERT INTO Salarie (SalNom,SalPrenom) VALUES (@nom,@prenom) :

```
CREATE PROCEDURE [dbo].[ajoutSalarie]
    @nom nvarchar(20),
    @prenom nvarchar(20)
AS
    INSERT INTO Salarie (SalNom,SalPrenom) VALUES (@nom,@prenom)
RETURN 0
```

- L'enregistrer dans la base (*Update, Update Database*) ;
- Utiliser la procédure stockée dans le C# à la place du SQL, créant les deux paramètres :

```
try
{
    SqlCommand sqlCmd = new SqlCommand("ajoutSalarie", accesSGBD.connexion);
    sqlCmd.CommandType = CommandType.StoredProcedure;
    SqlParameter salNom = sqlCmd.Parameters.Add("@nom", SqlDbType.NVarChar, 20);
    salNom.Value = unSalarie.nom;
    SqlParameter salPrenom = sqlCmd.Parameters.Add("@prenom", SqlDbType.NVarChar, 20);
    salPrenom.Value = unSalarie.prenom;
    sqlCmd.ExecuteNonQuery();
}
```

- Vérifier le bon fonctionnement

Pour les deux dernières méthodes (*modifierSalarie* et *supprimerSalarie*), la démarche est similaire à celle d'ajouterSalarie :

- Procédure stockée *modificationSalarie* :

```
CREATE PROCEDURE [dbo].[modificationSalarie]
    @no int,
    @nom nvarchar(20),
    @prenom nvarchar(20)
AS
    UPDATE Salarie SET SalNom=@nom, SalPrenom=@prenom WHERE SalNo=@no
RETURN 0
```

- Code C# utilisant la procédure stockée :


```
try
{
    SqlCommand sqlCmd = new SqlCommand("modificationSalarie", accesSGBD.connexion);
    sqlCmd.CommandType = CommandType.StoredProcedure;
    SqlParameter salNo = sqlCmd.Parameters.Add("@no", SqlDbType.Int);
    salNo.Value = unSalarie.no;
    SqlParameter salNom = sqlCmd.Parameters.Add("@nom", SqlDbType.NVarChar, 20);
    salNom.Value = unSalarie.nom;
    SqlParameter salPrenom = sqlCmd.Parameters.Add("@prenom", SqlDbType.NVarChar, 20);
    salPrenom.Value = unSalarie.prenom;
    sqlCmd.ExecuteNonQuery();
}
```

- Procédure stockée *suppressionSalarie* :

```
CREATE PROCEDURE [dbo].[suppressionSalarie]
    @no int
AS
    DELETE FROM Salarie WHERE SalNo=@no
RETURN 0
```

- Code C# utilisant la procédure stockée :

```
try
{
    SqlCommand sqlCmd = new SqlCommand("suppressionSalarie", accesSGBD.connexion);
    sqlCmd.CommandType = CommandType.StoredProcedure;
    SqlParameter salNo = sqlCmd.Parameters.Add("@no", SqlDbType.Int);
    salNo.Value = unSalarie.no;
    sqlCmd.ExecuteNonQuery();
}
```

2. Classe AbsenceDAO

Une seule méthode à adapter : *lireLesAbsencesDeSalarie*. Utilisation d'une procédure stockée paramétrée :

- Procédure stockée *listeAbsencesDeSalarie* :

```
CREATE PROCEDURE [dbo].[listeAbsencesDeSalarie]
    @noSalarie int
AS
    SELECT * FROM Absence, Motif WHERE AbsNoMotif=MotNo AND AbsNoSalarie=@noSalarie
RETURN 0
```

- Code C# appelant la procédure stockée :

```
try
{
    SqlCommand sqlCmd = new SqlCommand("listeAbsencesDeSalarie", accesSGBD.connexion);
    sqlCmd.CommandType = CommandType.StoredProcedure;
    SqlParameter salNo = sqlCmd.Parameters.Add("@noSalarie", SqlDbType.Int);
    salNo.Value = unSalarie.no;
    SqlDataReader sqlDataReader = sqlCmd.ExecuteReader();
    try
```

3. Classe MotifDAO

Une seule méthode à adapter : *lireLesMotifs*. Utilisation d'une procédure stockée sans paramètre :

- Procédure stockée *listeMotifs* :


```
CREATE PROCEDURE [dbo].[listeMotifs]
AS
    SELECT * FROM Motif
RETURN 0
```

- Code C# appelant la procédure stockée :

```
try
{
    SqlCommand sqlCmd = new SqlCommand("listeMotifs", accesSGBD.connexion);
    sqlCmd.CommandType = CommandType.StoredProcedure;
    SqlDataReader sqlDataReader = sqlCmd.ExecuteReader();
    try
```

V. Conclusion

Toujours un peu plus de travail, mais :

- De la sécurité dans les requêtes SQL (requêtes maîtrisées, pas d'injection SQL possible, ...) ;
- Adapté aux gros projets, avec répartition des tâches / rôles ;
- Le code est plus efficace (procédures stockées précompilées sur le SGBD) ;

Obtenu au chapitre précédent :

- La maintenance / évolution est facilitée ;
- La migration vers un autre support de stockage (fichiers XML, autre SGBD, ...) est envisageable ;
- Sécurité dans le code (erreur SQL interceptée) ;

VI. PPE

Vous êtes chargé de réaliser la phase 2 en recourant aux procédures stockées.

Prenez le projet de ce chapitre, et complétez-le avec le code déjà réalisé du chapitre précédent, que vous adapterez.

Gardez bien des projets distincts pour chaque chapitre.